

# Basic Commands in Linux

---

## 1 Present working directory:

```
ubuntu@dheen:~$ pwd
```

⇒ /home/ubuntu

## 2 Version:

```
ubuntu@dheen:~$ uname -a
```

⇒ version will appear

## 3 Username:

```
ubuntu@dheen:~$ whoami
```

⇒ ubuntu

## 4 Clearing the screen:

```
ubuntu@dheen:~$ clear
```

(ctrl + L)

⇒ Terminal will be cleared.

## 5 Viewing command history:

```
ubuntu@dheen:~$ history
```

⇒ Total history can be viewed.

# Creating Directory and Files in Linux

---

## 1 Make directory:

```
ubuntu@dheen:~$ mkdir data
```

⇒ Data file will be created.

## 2 Entering into a directory:

```
ubuntu@dheen:~$ cd data/
```

⇒ We will be enter into 'data' file.

## 3 Creating a text file:

```
ubuntu@dheen:~/data$ vi hello
```

⇒ "hello" text file will be created.

### Text file creation

⇒ In the editor, you have to give "i" for inserting a text.

⇒ Enter some text.

⇒ Give Escape button.

⇒ For saving a file ⇒ :wq → save

⇒ :q → quit

## 4 List of files:

```
ubuntu@dheen:~/data$ ls
```

⇒ hello.txt.

## 5 Quit file without saving:

- In the editor → give Escape.

```
:q!
```

## Installing a package: (Linux)

```
ubuntu@dheen:~/data$ sudo apt-get install vim
```

`sudo` ⇒ root user

`apt-get` ⇒ getting any application

`vim` ⇒ upgrading vi editor

## Auto-completing a command:

- Press **Tab** for auto-complete a file.
- (If we create two or more files).

## In data directory:

```
hello.txt      hello1      hello.py
```

- If we give ⇒ `vi hello`
  - output ⇒ `hello.txt` `hello1` `hello.py`
- If we give ⇒ `viP hello`
  - output ⇒ `hello.txt` `hello.py`
- If we give ⇒ `VP hello`
  - output ⇒ `hello.txt` `hello.py`

## 6 Nano Text Editor:

```
ubuntu@dheen:~/data$ nano test.txt
```

```
ubuntu@dheen:~/data$ ls
```

⇒ Htest.py hello.txt hello1 hello.ho test.txt

- Nano editor is simple, easy to use.
- 

## 7 Creating a dummy file:

```
ubuntu@dheen:~/data$ touch foo.txt
```

(After ls)

⇒ Htest.py hello.txt hello1 foo.txt test.txt

---

## 8 File manipulation:

- remove (deleting) a file: 

1 

```
ubuntu@dheen:~/data$ rm foo.txt
```

(After ls)

⇒ Htest.py hello.txt hello1 test.txt

2 

```
ubuntu@dheen:~/data$ ls *.txt
```

⇒ hello.txt test.txt

3 

```
ubuntu@dheen:~/data$ ls he*
```

⇒ hello.txt hello1

4 

```
ubuntu@dheen:~/data$ rm *
```

⇒ Total files be deleted.

# Viewing and Copying a File

---

```
ubuntu@dheen:~/data$ vi hello.txt
```

(hello.txt will some text is created).

## 1 Displaying content in command prompt:

```
ubuntu@dheen:~/data$ cat hello.txt
```

// cat

⇒ This is a file that is created.

---

## 2 Copying a file: ❌

```
~/data$ cp hello.txt new-hello.txt
```

// cp

```
~/data$ ls
```

⇒ hello.txt      new-hello.txt

---

## 3 Renaming a file: ❌

```
~/data$ mv hello.txt demo.txt
```

// mv

```
~/data$ ls
```

⇒ demo.txt      new-hello.txt

```
~/data$ cat demo.txt
```

⇒ This is a file that is created.

## 4 Copying a file with 'cat' command:

Append symbol ( >> ) :

```
ubuntu@dheen:~/data$ cat demo.txt >> nfile.txt
```

(After ls)

⇒ demo.txt new-hello.txt nfile.txt

(Copies the text from one file to another).

---

## 5 Printing texts in prompt:

1

```
~/data$ echo "dheen"
```

// echo

⇒ dheen

---

2

```
~/data$ echo "dheen" >> nfile.txt
```

// store in file

(The text will be stored in file).

---

3

```
~/data$ echo "mohamed" >> nfile.txt
```

// append in file

```
~/data$ cat nfile.txt
```

⇒ This is a file that is created

dheen

mohamed

## 1. ls and cd commands:

```
ubuntu@dheen:~$ ls
```

```
⇒  untitled.pnb    data    print_time.py
    airflow        data.log  retal.txt
```

### Shortcut for "home/ubuntu":

```
ubuntu@dheen:~$ cd data/
```

```
ubuntu@dheen:~/data$ data.log
```

- No such file directory

```
: ~/data $ cd .      N → Home directory
```

```
: ~$ vi data.log     → cd ~
```

Give ⇒ "This is a log file" and save it.

```
⇒ : ~$ cd data/
```

```
: ~/data $ cd ..
```

```
: ~$ cd data
(home/ubuntu/data.log)
```

"This is a log file"

```
* ⇒ : ~/data  ~ /data.log  ( / ~ )
    =
    "This is a log file".
```

## 2. Creating nested Directory:

- If we want to create directory (or) file in the file that doesn't exist (or) created
- we have to create it one by one into a file.
- Instead we use "-p".

```
ubuntu@dheen:~$ pwd
```

```
/* // -p
```

```
/home/ubuntu
```

```
ubuntu@dheen:~$ mkdir -p abc/test/demo
```

```
~$ cd abc/
```

```
~$ ls
```

```
⇒ test
```

## 3. Going Back from a directory:

```
ubuntu@dheen:~$ cd abc/test/demo
```

```
ubuntu@dheen:~$ pwd
```

```
/home/ubuntu/abc/test/demo
```

```
ubuntu@dheen:~$ cd ..      (home/ubuntu/abc/test)
```

```
ubuntu@dheen:~$ cd ../../  (home/ubuntu)
```

```
ubuntu@dheen:~$ cd -      (will go to previous cd command)
```

```
/home/ubuntu/abc/test
```

## 4. Copying a file with 'cat' command:

Append symbol ( >> ) :

```
ubuntu@dheen:~/data$ cat demo.txt >> nfile.txt
```

(After ls)

```
⇒ demo.txt  new-hello.txt  nfile.txt
```

(Copies the text from one file to another).

## 5. Printing texts in prompt:

```
① ~ /data$ echo "dheen"      // echo
    ⇒ dheen
```

```
② ~ /data$ echo "dheen" >> nfile.txt  // store in file
(The text will be stored in file).
```

```
③ ~ /data$ echo "mohamed" >> nfile.txt  // append in file
    ~ /data$ cat nfile.txt
    ⇒ This is a file that is created
       dheen
       mohamed
```

# List Functionalities: (ls)

## 1 Basic ls command:

```
ubuntu@dheen:~$ ls
```

```
abc          data.log     print_time.py
airflow      data         hadoop2-din
```

## 2 ls options:

| Option | Description                               |
|--------|-------------------------------------------|
| -l     | long list format                          |
| -s     | shows file size in blocks                 |
| -t     | sorts by modification time (newest first) |
| -r     | reverses the sorting order                |
| -S     | sorts by file size (largest first)        |

```
ubuntu@dheen:~$ ls -lstrS
```

Output => **total 885076**

```
404908  -rw-r--r--  1  ubuntu  ubuntu  1414  May 31  abc
189406  drwxr-xr-x  3  ubuntu  ubuntu   501  Jun 10  logs
...
```

## 3 ls -a (shows hidden files):

```
ubuntu@dheen:~$ ls -a
```

(shows the hidden files)

```
.          bash-history  .profile     print_time.py
..         .bashrc       data.log     hadoop2-din
abc        data         .
```

### Note:

- **-a** option shows hidden files (files starting with dot ".")
- Without **-a**, hidden files are not shown.

| Hidden files  | Description                |
|---------------|----------------------------|
| .bash_history | Bash command history       |
| .profile      | User profile settings      |
| print_time.py | Python file (visible file) |
| .bashrc       | Bash configuration file    |
| data.log      | Log file (visible file)    |
| hadoop2-din   | Directory (visible)        |

## 4 .bashrc file usage

⇒ In .bashrc file, if we install any Python on Java we will give environment variables in it.

Opening .bashrc file:

```
ubuntu@dheen:~$ vi .bashrc
```

Executing .bashrc file:

```
1 ubuntu@dheen:~$ source .bashrc
```

```
2 ubuntu@dheen:~$ . .bashrc
```

## 5 List manipulations

1 Basic ls command:

```
ubuntu@dheen:~$ ls
```

```
abc          data.log     print_time.py
airflow      data        hadoop
```

2 List options:

| Option | Description                               |
|--------|-------------------------------------------|
| -l     | long list format                          |
| -s     | shows file size in blocks                 |
| -t     | sorts by modification time (newest first) |
| -r     | reverses the sorting order                |
| -S     | sorts by file size (largest first)        |

```
ubuntu@dheen:~$ ls -ltrS
```

Output => **total 885076**

```
404908 -rw-r--r-- 1 ubuntu ubuntu 1414 May 31 abc
189406 drwxr-xr-x 3 ubuntu ubuntu 501 Jun 10 logs
...
```

3 ls -a (shows hidden files):

```
ubuntu@dheen:~$ ls -a
```

(shows the hidden files)

```
.      bash_history .profile print_time.py
..     .bashrc      data.log  hadoop2-din
abc    data          .
```

List manipulations (cont.):

```
1 :~$ ls * .py
```

⇒ print\_time.py

```
2 :~$ ls data*
```

⇒ data.log data

 **Note:**

- \* is a wildcard.
- \* matches zero or more characters.

List manipulations (cont.):

```
1 :~$ ls * .py
```

⇒ print\_time.py

```
2 :~$ ls data*
```

⇒ data.log data

| Hidden File / Item | Description                |
|--------------------|----------------------------|
| .bash_history      | Bash command history       |
| .profile           | User profile settings      |
| print_time.py      | Python file (visible file) |
| .bashrc            | Bash configuration file    |
| data.log           | Log file (visible file)    |
| hadoop2-din        | Directory (visible)        |

# Hardlink and Softlink

## 1. Create a file:

```
:~$ echo "hello linux" >> original.txt # write content
:~$ cat original.txt # view content
⇒ hello linux
:~$ ls -lstr # list with details
⇒ total 4
 4 -rw-r--r-- 1 ubuntu Jan 9 original.txt
```

## 2. Hardlink:

```
:~$ ln original.txt team2-data.txt # create hardlink
:~$ ls -lstr # list with details
⇒ Total 8
 4 -rw-r--r-- 2 ubuntu ..... team2-data.txt
 4 -rw-r--r-- 2 ubuntu ..... original.txt
```

- ⇒ If we update anything in "Original.txt", it applies (also) on "team2-data.txt".
- ⇒ If we delete one file, another file will exist.

## 3. Softlink (Symbolic link):

```
:~$ ln -s original.txt new-data.txt # create softlink
:~$ ls -lstr # list with details { (In gitlab i deleted original file but copy file didn't get deleted)
⇒ Total 4
 4 -rw-r--r-- 1 ubuntu ..... original.txt
 0 lrwxrwxrwx 1 ubuntu ..... new-data.txt -> original.txt
```

### Important Notes:

- ★ If we update in one file, it also update another but if we delete source file, child file also be deleted.
- ★ Hardlink and Softlink are useful for **backup**, **sharing files**, and **saving space**.

### Quick Summary

- Hardlink → points to the same data (same inode). Deleting one name **does not** delete the data.
- Softlink → points to the path of the original file. Deleting original file **breaks the link**.

### Hardlink vs Softlink

| Feature          | Hardlink         | Softlink            |
|------------------|------------------|---------------------|
| Inode            | Same inode       | Different inode     |
| Link Count       | Increases        | No effect           |
| Size             | Same as original | 0 (shows link only) |
| Cross Filesystem | Not possible     | Possible            |
| Delete Original  | Data deleted     | Link broken         |
| Type             | File link        | Symbolic link       |

## Background Run

Running a program / command in the **background** without stopping the terminal.

## Example – Python code

```
import time

a = 0
b = 5

while True:
    a += b
    print(f"Updated value of a: {a}")
    time.sleep(3)
```

For every 3 seconds, a value will be printed.

## 1 Running Python in background

```
~$ vi data.py
```

# Create / edit Python file

```
~$ python3 data.py &
```

# Run in background



Python3 will be installed by default in Linux.

### Output:

```
Updated value of a: 5
Updated value of a: 10
...
```

## ★ Remember:

- If we run this line (command), it will print the values of output, but it is not permanent.
- If we close the Linux system (or) if someone stops the environment run, it will be deleted.
- The process will stop and will be deleted, we have to run every time.

## 2 nohup command:

```
:~$ nohup python3 data.py >> data.py.log &
```

↓  
can't stop the run  
(if system is closed)

↓  
will be saved to  
this file

## 3 Background checker (top command):

```
:~$ top
```

⇒ All running applications will be displayed  
in the task manager.

| PID | USER  | PR | NI | RES  | %CPU  | TIME+ | COMMAND |
|-----|-------|----|----|------|-------|-------|---------|
| 522 | deven | 20 | 0  | 4681 | 37.00 | 0.3   | python3 |

## 4 Process status: (ps)

```
:~$ ps
```

⇒ Shows current running processes.

| PID    | TTY   | TIME+ | CMD     |
|--------|-------|-------|---------|
| 161616 | pts/2 | 00:00 | python3 |

## 5 Detailed status:

```
:~$ ps -aux
```

|       |        |     |     |      |      |   |   |       |         |                     |
|-------|--------|-----|-----|------|------|---|---|-------|---------|---------------------|
| deven | 161616 | 0.0 | 0.3 | 4681 | 3700 | ? | S | 00:00 | python3 | data-processing.lpy |
|-------|--------|-----|-----|------|------|---|---|-------|---------|---------------------|

00:00 → start time

python3 → command being executed

data-processing.log → argument / file name

a → show process from all users

u → show the user who owns the process

x → show processes attached to terminal

## 6 Displaying the file that contains (nohup outputs):

```
:~$ cat data.py.log
```



### Tip:

- **nohup** command allows the process to continue running even after logout.
- Use **&** to run process in background and save output to a file.

## 7 Displaying the file (Running file):

```
:~$ tail -f data-process.log # tail -f
```

⇒ It will give last 10 lines and displays  
running file (live).

```
:~$ tail -20 data-py.log
```

⇒ It will give last 20 lines (live).

### Note:

- **tail -f** → follows the file and shows updates.
- **tail -n N** → shows last N lines once.

## 8 Killing the run:

```
:~$ kill -9 115966 (PID of the file)
```

↓  
kill forcefully.



### Note:

kill -9 sends SIGKILL signal and stops the process immediately.

## 9 Pattern matching:

```
:~$ ps -aux | grep data-processing.py
```

⇒ Twisted command (one command binds with another)

```
ps -aux | grep data-processing.py
```

↓  
gives detailed status  
of all processes

↓  
gives only the file  
that is matching

|        |        |     |     |      |      |   |   |       |         |                    |
|--------|--------|-----|-----|------|------|---|---|-------|---------|--------------------|
| ubuntu | 167432 | 0.0 | 0.3 | 4681 | 3700 | ? | S | 00:00 | python3 | data-processing.py |
|--------|--------|-----|-----|------|------|---|---|-------|---------|--------------------|

### Summary of Commands

| Command        | Purpose                                     | Example                                |
|----------------|---------------------------------------------|----------------------------------------|
| nohup          | Run process in background even after logout | nohup python3 data.py >> data.py.log & |
| &              | Send process to background and save output  | ... &                                  |
| top            | Display all running processes               | top                                    |
| ps             | Show current running processes              | ps                                     |
| ps -aux        | Show detailed status of all processes       | ps -aux                                |
| tail -f        | Show last 10 lines (live updates)           | tail -f file.log                       |
| tail -n N      | Show last N lines once                      | tail -20 file.log                      |
| kill -9 PID    | Kill a process forcefully                   | kill -9 115966                         |
| ps -aux   grep | Filter output based on pattern (matching)   | ps -aux   grep file_name               |

# Downloading Image (or) File from internet

## 1 wget

### Example:

⇒ Copy the link of the image in website

```
:~$ wget "https://....." # wget
```

```
:~$ ls
```

⇒ linux.jpg

Alias in Linux: ✓



### Notes:

- This command downloads the file from the given URL and saves it in the current directory.

## 1 For normal command, we have to give repeated the same path and same command.

⇒ If we use "Alias" it will become simpler.

### Example:

```
:~$ alias dheen='cd /home/ubuntu/abc.....'
```

```
:~$ dheen
```

⇒ It will go to the path directly.



### Note:

- Alias is a shortcut for a command or path.
- It saves time and makes work easier.

## 2 If we want to use Alias name permanently, we have to save it in "bashrc" file and to execute it.



### How to make Alias permanent?

1. Open the file

```
:~$ vi ~/.bashrc
```

2. Add alias at the end of the file

```
alias dheen='cd /home/ubuntu/abc.....'
```

3. Save and close the file.

4. Execute the file to apply changes

```
:~$ source ~/.bashrc
```

5. Now **alias** name can be used anywhere.

## ★ Summary:

- **wget** → Download file from internet using URL.
- **alias** → Shortcut for long command or path.
- Temporary alias → Works only in current terminal session.
- Permanent alias → Save in `~/.bashrc` and execute.

# Data - Management

## 1 Disk free-space:

```
:~$ df -h
```

# df -h

- ⇒ df → disk
- ⇒ -h → human readable format
- ⇒ Shows file-system disk space usage.

| File-system    | Size | Used | Mounted on |
|----------------|------|------|------------|
| /dev/nvme0n1p2 | 8.9G | 4.0K | / (root)   |

## 2 Particular file usage:

```
:~$ du -sh scripts/
```

# du -sh

- ⇒ du → disk usage
- ⇒ -sh → summary / human-readable
- ⇒ Shows total size of "scripts/" directory.
- ⇒ 12K scripts/

## RAM - usage:

### 1 Display with mb/gb: X

```
:~$ free -m
```

 ⇒ mb details

```
:~$ free -g
```

 ⇒ gb details

|      | total | used | free | cache |
|------|-------|------|------|-------|
| Mem: | 9798  | 629  | 6390 | 0.    |

### 2 Free-up cache:

```
:~$ sudo sh -c "sync; echo 3 > /proc/sys/vm/drop_caches"
```

#### What it does:

- sync → flushes all file system buffers to disk.
- echo 3 > /proc/sys/vm/drop\_caches → clears pagecache, dentries and inodes.

### Summary

- df -h → Shows disk space usage in human readable format.
- du -sh <dir/file> → Shows total size of a directory/file.
- free -m / free -g → Shows RAM usage in MB / GB.
- sudo sh -c "sync; echo 3 > /proc/sys/vm/drop\_caches" → Frees up memory cache.

### Quick Notes

- Use df -h to check disk space.
- Use du -sh to check size of a file or directory.
- Use free -m or free -g to check RAM usage.
- Clear cache only if system is slow due to memory pressure.

# Zip and Unzipping a File

⇒ “zip” and “unzip” commands.

## 1 Zipping a file:

```
:~$ zip file.zip scripts/
```

⇒ If zip is not installed.

```
:~$ sudo apt-get install zip
```

⇒ If any error comes for installing

we ⇒ 

```
:~$ sudo apt-get update
```



### What it does?

- Creates a zip archive of the given file / directory.

### After installation:

```
:~$ zip -r file1.zip scripts/
```



**-r (recursive)**

Include all files and subdirectories (contents of folder)



**file1.zip**

Destination zip file name



**scripts/**

Source file / directory to zip

## 2 Unzipping the file:

```
:~$ unzip file1.zip
```

```
:~$ ls -lstr | tail -f
```



### What it does?

- **unzip file1.zip** → Extracts the zip archive.
- **ls -lstr | tail -f** → Shows the latest files and directories live.

### Explanation:

- **file1.zip** → Zip archive file
- **scripts/** → Extracted folder (contents of zip)

## Summary

### 1. Zipping

- **zip <zipname.zip> <file/folder>**  
→ Create zip archive.
- **-r** → Include all files and subdirectories recursively.
- Install zip (if not installed):  
`sudo apt-get install zip`
- Update (if any error):  
`sudo apt-get update`

### 2. Unzipping

- **unzip <zipfile.zip>** → Extract the zip file.
- **ls -lstr | tail -f** → Show the latest files (live).

### Notes:

- **zip** → Compress files / folders.
- **unzip** → Extract files from zip archive.

# TAR – Archiving and Extracting Files

## 1 Creating (Archiving) a File

# **tar -cvzf**

```
:~$ tar -cvzf sample.tar scripts/
```

- c** → compress archive ✓
- z** → gzip compression ✓
- v** → verbose (show files being archived) ✓
- f** → use archive file name ✓

**scripts/** → directory to archive  
**scripts/wrapper\_script.sh** → file to archive

### Option Summary

- c** Create a new archive
- z** Compress the archive (gzip)
- v** Verbose output (show files)
- f** Use archive file name

## 2 Deleting (Removing) Files or Directories

```
:~$ rm -rf scripts/
```

- rm** → remove (delete)
- rf** → recursive force (deletes folders and their contents)

### Note:

- Use **-rf** carefully. It removes the folder and all its contents permanently.

## 3 Extracting (Untarring) a File

# **tar -zxvf**

```
:~$ tar -zxvf sample.tar
```

- x** → extract (untar)

```
:~$ ls -lstr | tail -f
```

Show the latest files and directories (live).

### Option Summary (untar)

- z** Decompress (gzip)
- x** Extract files from archive
- v** Verbose (show files)
- f** Use archive file name

| Permissions | Links | Owner  | Size | Name       |
|-------------|-------|--------|------|------------|
| drwxr-xr-x  | 2     | ubuntu | ---- | scripts/   |
| -rw-r--r--  | 1     | ubuntu | ---- | sample.tar |

### Quick Summary

#### Create Archive (tar)

**tar -cvzf** <archive.tar.gz> <file/dir>

- c** : create archive
- z** : compress (gzip)
- v** : verbose (show files)
- f** : specify archive file name



#### Delete (rm)

**rm -rf** <file/dir>

- rm** : remove (delete)
- rf** : recursive force (no prompt)

Use with acution!



#### Extract Archive (untar)

**tar -zxvf** <archive.tar.gz>

- z** : decompress (gzip)
- x** : extract
- v** : verbose (show files)
- f** : specify archive file name



- Use **-v** to see what is being archived / extracted.
- Always check files after extraction using **ls**.
- tar** is useful for backup, transfer and compression.

## Counting the words, characters and lines.

```
~$ vi sample.txt
```

Example content:

```
linux
user is good
I am a person
(save it)
```

### Note:

- **wc** stands for "word count".
- It is used to count:
  - Words
  - Characters
  - Lines

| No. | Command                         | Output Example                                                                                   | Explanation                                                                                                                     | Counts                                                                                           |                  |
|-----|---------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|------------------|
| 1   | <pre>~\$ wc sample.txt</pre>    | <pre>3      8      42 sample.txt</pre> <p>↓       ↓       ↓<br/>(Words) (Characters) (Lines)</p> | <ul style="list-style-type: none"> <li>• Default wc command</li> <li>• Prints number of words, characters and lines.</li> </ul> | <ul style="list-style-type: none"> <li>• Words</li> <li>• Characters</li> <li>• Lines</li> </ul> | # wc [file name] |
| 2   | <pre>~\$ wc -w sample.txt</pre> | <pre>8 sample.txt</pre>                                                                          | <ul style="list-style-type: none"> <li>• Gives the number of words in the file.</li> </ul>                                      | <ul style="list-style-type: none"> <li>• Words</li> </ul>                                        | # wc -w          |
| 3   | <pre>~\$ wc -l sample.txt</pre> | <pre>3 sample.txt</pre>                                                                          | <ul style="list-style-type: none"> <li>• Gives the number of lines in the file.</li> </ul>                                      | <ul style="list-style-type: none"> <li>• Lines</li> </ul>                                        | # wc -l          |
| 4   | <pre>~\$ wc -c sample.txt</pre> | <pre>42 sample.txt</pre>                                                                         | <ul style="list-style-type: none"> <li>• Gives the number of characters in the file.</li> </ul>                                 | <ul style="list-style-type: none"> <li>• Characters</li> </ul>                                   | # wc -c          |

### Summary

| Command      | What it does                       | Example Output    |
|--------------|------------------------------------|-------------------|
| wc [file]    | Counts words, characters and lines | 3 8 42 sample.txt |
| wc -w [file] | Counts number of words             | 8 sample.txt      |
| wc -l [file] | Counts number of lines             | 3 sample.txt      |
| wc -c [file] | Counts number of characters        | 42 sample.txt     |

# Sorting the Lines

# sort

## 1. Sort in Alphabetical Order (Default)

Example (Alphabetical order):

Mango  
zebra  
Apple

```
:$ sort file.txt
```



Apple  
Mango  
zebra

Sort with Numerical Order:

# sort -n

Example:

9  
11  
1  
24

```
:$ sort -n file.txt
```



1  
9  
11  
24

If we give **sort -n** file.txt it will give in reverse order (numerical ascending order).

## Head & Tail Commands

(If we have to display first 10 or last 10 lines)

| Command                           | What it does                      | Example                              | Output / Explanation                          | Tag                      |
|-----------------------------------|-----------------------------------|--------------------------------------|-----------------------------------------------|--------------------------|
| <pre>:\$ vi file.txt</pre>        | Display the content of the file.  | (Opens the file in <b>vi</b> editor) | Shows all lines in file.                      | # vi                     |
| <pre>:\$ head file.txt</pre>      | Display first 10 lines (default). | (Shows first 10 lines)               | First 10 lines of the file will be displayed. | # head                   |
| <pre>:\$ tail file.txt</pre>      | Display last 10 lines (default).  | (Shows last 10 lines)                | Last 10 lines of the file will be displayed.  | # tail                   |
| <pre>:\$ head -n 5 file.txt</pre> | Display first 5 lines.            | (Shows first 5 lines)                | First 5 lines will be displayed.              | # head -n 5<br># tail -5 |
| <pre>:\$ tail -n 5 file.txt</pre> | Display last 5 lines.             | (Shows last 5 lines)                 | Last 5 lines will be displayed.               | # tail -n 5<br># tail -5 |

### Sorting Summary

| Command                     | Meaning                            | Result                  |
|-----------------------------|------------------------------------|-------------------------|
| <pre>sort file.txt</pre>    | Sort alphabetically                | Apple<br>Mango<br>zebra |
| <pre>sort -n file.txt</pre> | Sort numerically (ascending order) | 1<br>9<br>11<br>24      |

### Head & Tail Summary

| Command                       | Meaning             | Displays       |
|-------------------------------|---------------------|----------------|
| <pre>vi file.txt</pre>        | Open file in editor | Entire file    |
| <pre>head file.txt</pre>      | Show first 10 lines | First 10 lines |
| <pre>tail file.txt</pre>      | Show last 10 lines  | Last 10 lines  |
| <pre>head -n 5 file.txt</pre> | Show first 5 lines  | First 5 lines  |
| <pre>tail -n 5 file.txt</pre> | Show last 5 lines   | Last 5 lines   |

# Pattern Matching & Finding Matches

## 1. Grep Command (grep)

# grep

Used to **find / identify** a file or text lines that are getting **matched** with the given pattern (string or expression).

### Example:

\* Create two files with some log texts

```
File: log1.txt
[INFO] Server started
[DEBUG] Connection success
[ERROR] File not found
[INFO] Job completed
[WARNING] Low memory
[ERROR] Timeout reached
```

```
File: log2.txt
[INFO] Server started
[DEBUG] Connection success
[ERROR] File not found
[INFO] Job completed
[WARNING] Low memory
[ERROR] Timeout reached
```

### Note:

- **grep** searches for the given pattern in files and prints the matching lines.
- Pattern matching is **CASE SENSITIVE** by default.

Save the files with file names: `log1.txt` and `log2.txt`

## 2. Common grep Usage

| No. | Command                                                                                      | Explanation                                                    | Output / Example                                                            | Tag                   |
|-----|----------------------------------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------|
| 1   | <code>grep ERROR log2.txt</code><br>(Searches for "ERROR")                                   | Displays lines that contain the pattern "ERROR".               | <code>[ERROR] File not found</code><br><code>[ERROR] Timeout reached</code> | # grep [find text]    |
| 2   | <code>grep -i error log1.txt</code><br>(Searches for "error", case insensitive)              | Displays lines that match "error" (case insensitive).          | <code>[ERROR] File not found</code><br><code>[ERROR] Timeout reached</code> | # grep -i [find text] |
| 3   | <code>grep -i "error" log1.txt</code><br>(Searches for exact word "error", case insensitive) | Displays lines that match the word "error" (case insensitive). | <code>[ERROR] File not found</code><br><code>[ERROR] Timeout reached</code> | # grep -i [find text] |

### Tips:

- By default, **grep** is case sensitive.
- Use **-i** to ignore case (case insensitive search).
- You can search in multiple files also: `grep pattern file1.txt file2.txt`

# GREP – Advanced Usage

## 3 `grep -v ERROR log1.txt`

# `grep -v`

(It will give inverted) except that lines

```
[INFO] ...  
...  
[WARNING] ...
```

**-v** → Invert match (show lines that do **NOT** match the pattern)

## 4 `grep WARNING *.txt`

# `grep *[pattern]`

(gives matched lines within the file matched)

```
log1.txt: [WARNING] ....  
log2.txt: [WARNING] ....
```

**\*** → Matches any file names (wildcard). Searches all matching files.

## 5 `grep WARNING *.txt | tail -f`

# `-f` → file

(gives last 10 lines (records))

**tail -f** → Follow the file and show last 10 lines (records).

## 6 `ls -lstr | tail -f`

# with `-lstr`

(gives last 10 files)

**-lstr** → long listing, sort by time (newest first), then show last 10 lines.

⇒ If we are having a file that contains matched item

⇒ In that directory itself having matched items

### Example Directory Structure

```
/logs  
├─ data  
├─ sample  
├─ demo  
└─ *.txt ✓
```

## 7 `grep -r "Timeout" ~ /logs`

# `-r`

```
/home/ubuntu/logs/test/list : Timeout  
/home/ubuntu/logs/data.txt : Timeout
```

**-r** → Recursive search in directories. Searches in all files under /logs.

## 8 `grep "^[ERROR]" log1.txt`

# `-n`

(gives the line that start with pattern)

**^** → Matches the beginning of the line. Shows lines that start with **[ERROR]**.

### Quick Reference

- **-i** → Case insensitive search
- **-v** → Invert match (exclude matching lines)
- **\*** → Wildcard (match any file name)
- **-f** → Follow file (tail)
- **-r** → Recursive search in directories
- **^** → Match beginning of line

### Common Pattern Examples

| Pattern     | Meaning                           |
|-------------|-----------------------------------|
| ERROR       | → Lines containing the word ERROR |
| ^[ERROR]    | → Lines starting with [ERROR]     |
| WARN.*      | → Lines starting with WARN        |
| .*timeout.* | → Lines containing timeout        |

# SECURE SHELL (SSH)

# openssh-server

★ Used to connect one server to another server.

(We can connect our friend's Ubuntu system with our server using SSH)

## 1 Installation:

install

```
~$ sudo apt-get install openssh-server
```

(SSH will be installed)

```
~$ sudo apt-get update openssh-server
```

(Updates the SSH)

```
~$ ifconfig
```

 → command gives details about our server.

(IP also) ✓

## 2 Entering into another server: ✓

```
~$ ssh 192.20.124.118
```

★ after password, we can login into our friend's system.

### Tips:

- SSH uses port 22 by default.
- Make sure SSH service is running on both systems.
- Use `ifconfig` / `ip a` to get IP.

## 3 Login using user-name:

```
~$ hostname
```

 ⇒ dheen

```
~$ ssh ubuntu@dheen
```

(Login to user ubuntu whose name is dheen)

# SECURE COPY (SCP)

# scp (secure copy)

★ Used to copy one file to another server.

★ From server to server copying.

Example:

```
~$ scp /home/ubuntu/logs/job.bile ubuntu@dheen:/home/ubuntu/copy.bile
```

(It will ask password and it will copy)

### Format (with IP):

```
scp <source_file> <user>@<ip>:<destination_path>
```

### (with host name):

```
scp <source_file> <user>@<hostname>:<destination_path>
```

| No. | Command                   | Description                         | Example / Output                           | Tag                  |
|-----|---------------------------|-------------------------------------|--------------------------------------------|----------------------|
| 1   | scp (copy file)           | Copy file from local to remote      | scp file.txt ubuntu@server:/home/ubuntu/   | # copy file          |
| 2   | scp (copy directory)      | Copy directory from local to remote | scp -r dir1 ubuntu@server:/home/ubuntu/    | # copy directory     |
| 3   | scp (remote to local)     | Copy file from remote to local      | scp ubuntu@server:/home/ubuntu/file.txt ./ | # download file      |
| 4   | scp (remote to local dir) | Copy directory from remote to local | scp -r ubuntu@server:/home/ubuntu/dir1 ./  | # download directory |

### Common SCP Options

- **-r** : recursive copy (for directory)
- **-P <port>** : specify port (default 22)
- **-C** : compress data while copying
- **-p** : preserve file permissions, time etc.

### Important Notes

- SCP uses SSH, so authentication is required.
- Make sure the remote path and file name are correct.
- Use **-r** option when copying directories.

# FIND COMMAND

## Find:

★ Used to **find a file**.

# find --name

1 `find . -name log1.txt`

(finds the file with name)

# find --name

2 `find . -name "*.txt"`

(find the file that ends with ".txt")

# --name "\*.txt"

3 `find . -type f -size +10M`

(type with file, size with >10MB)

# --type -size

4 `find . -mtime -1`

(finds the file that are created within 1 day)

# --mtime

5 `find . -empty`

(finds the empty file in whole system)

# --empty

6 `find . -name "*.tmp" -delete`

(deletes the file that ends with .tmp)

# --name "\*.tmp" -delete

| Feature                                                                                          | Find                                          | grep                                                 |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------------------|
|  Searches     | Files and directories                         | Text inside files                                    |
|  Used for     | Locating files by name, size, type, time etc. | Searching for matching lines (content) inside files. |
|  Search level | Filesystem level                              | File content level                                   |
|  Example      | <code>find . -name log1.txt</code>            | <code>grep "ERROR" log1.txt</code>                   |

### Common find Options

- name PATTERN** : search by name
- type f / d** : file (f) or directory (d)
- size +10M / -10M** : size greater than / less than
- mtime -1** : modified within 1 day
- empty** : find empty files/directories
- delete** : delete matched files

### Examples

- `find . -name "*.txt"`  
→ find all .txt files
- `find . -type f -size +10M`  
→ find files larger than 10MB
- `find . -empty`  
→ find empty files
- `find . -name "*.tmp" -delete`  
→ delete all .tmp files

### Notes:

- The dot (.) means current directory. Use / for root directory.
- You can combine multiple options with find for advanced searches.
- Be careful with -delete option.
- Use man **find** to see all available options.

# AWK (Command-line Tool)

AWK: (Alfred, Weinberger, Kernighan)

- ★ Processing columns in text.
- ★ Performs calculations.
- ★ Extract and format data from files.

## Examples:

```
~$ vi data.txt  
~$ cat data.txt
```

(Creating a file)

```
John  25  Developer  
Asha  30  Designer  
Ravi  28  Tester
```

1

```
~$ awk '{print $0}' data.txt
```

(same output will appear)

⇒

```
John  25  Developer  
Asha  30  Designer  
Ravi  28  Tester
```

awk '{print}'

2

```
~$ awk '{print $1}' data.txt
```

⇒

(column 1 data will be printed)

⇒

```
John  
Asha  
Ravi
```

'print \$1'

3

```
~$ awk '{print $1 , $3}' data.txt
```

⇒

(column 1 and column 3 data will be printed)

```
John    Developer  
Asha    Designer  
Ravi    Tester
```

'print \$1 , \$3'

## Common awk Print Variables

| Variable           | Meaning                                 |
|--------------------|-----------------------------------------|
| \$0                | Entire line                             |
| \$1, \$2, \$3, ... | Fields (columns) of the line            |
| NF                 | Number of fields (columns) in a line    |
| NR                 | Current line number                     |
| FNR                | Current line number in the current file |

## Useful awk Options

| Option        | Meaning                         |
|---------------|---------------------------------|
| -F 'delim'    | Set field separator (delimiter) |
| -v var=value  | Assign value to variable        |
| BEGIN { ... } | Execute before reading input    |
| END { ... }   | Execute after reading input     |

## Notes:

- Default field separator is any space or tab.
- Columns in data are separated by space or tab.
- Use quotes around awk programs: '{ }'.
- Very useful for text processing, data extraction and calculations.

4

```
awk '$2 > 27 {print $1, $3}' data.txt
```

# condition

⇒

```
Asha      Designer
Ravi      Tester
```

(Asha and Ravi age is 27 above)

5

```
awk 'printf "Name : %s | Age : %s | Role : %s\n",
      $1, $2, $3' data.txt
```

# formatted output

⇒

```
Name : John | Age : 25 | Role : Designer
Name : Asha | Age : 30 | Role : Developer
Name : Ravi | Age : 28 | Role : Tester
```

6

```
awk 'NR==3, NR==6 {print NR, $0}' employee.txt
```

# print line range

⇒

```
3  varun manager sales 50000
4
5
6  deepak clerk sales 23000
```

### Syntax of awk

```
awk [options] 'pattern {action}' input-file > output-file
```

- awk** - Starts the AWK text-processing program.
- [options]** - Control AWK behaviour (e.g., set field separator).
- pattern** - Specifies which lines to process.
- {action}** - Defines what to do with matching lines (usually print).
- input-file** - File that awk reads line by line.
- > output-file** - Redirects processed output into a file.

#### Common awk Options

| Option        | Meaning                         |
|---------------|---------------------------------|
| -F 'delim'    | Set field separator (delimiter) |
| -v var=value  | Assign value to variable        |
| BEGIN { ... } | Execute before reading input    |
| END { ... }   | Execute after reading input     |

#### Common awk Variables

| Variable           | Meaning                                 |
|--------------------|-----------------------------------------|
| \$0                | Entire line                             |
| \$1, \$2, \$3, ... | Fields (columns) of the line            |
| NF                 | Number of fields (columns) in a line    |
| NR                 | Current line number                     |
| FNR                | Current line number in the current file |

#### Notes:

- Default field separator is any space or tab.
- Columns in data are separated by space or tab.
- Use quotes around awk programs: '{ }'.
- Very useful for text processing, data extraction and calculations.

# CHANGE MODE (chmod)

18<sup>th</sup> April 2024

## chmod:

- ⇒ If we have to change the mode for any one to read , write and execute , we use chmod.
- ⇒ Permission based command.

## Team

## Who it is

|                                                                                   |        |   |                                                                        |
|-----------------------------------------------------------------------------------|--------|---|------------------------------------------------------------------------|
|  | User   | - | The user who <u>owns</u> the file.                                     |
|  | Group  | - | Members of the <u>file's group</u> .                                   |
|  | Others | - | <u>Everyone else</u> (excluding owner and group).                      |
|  | Root   | - | The <u>superuser</u> - <u>not affected</u> by regular file permission. |

## Patterns for the file:

### Example:

```
:$ ls -lstr
```

drwx n-x n-x

- d : directory
- rwx : permissions for User
- n-x : permissions for Group
- n-x : permissions for Others

drwx n-x n-x

- User permission  
(read , write , execute)
- Group  
(read , execute)
- Others  
(read , execute)
- Directory

drwx n-x n-x

(User , group , others)

rw → read , write , execute.

-rw- rw- r--

- file  
(read and write for user)
- Group  
(read and write)
- Others  
(read only)

## chmod Numeric Notation

| Permission        | Symbol | Value |
|-------------------|--------|-------|
| Read (r)          | r      | 4     |
| Write (w)         | w      | 2     |
| Execute (x)       | x      | 1     |
| No Permission (-) | -      | 0     |

## Examples (Numeric Form)

rw = 4 + 2 + 1 = 7  
rw- = 4 + 2 + 0 = 6  
r-x = 4 + 0 + 1 = 5  
r-- = 4 + 0 + 0 = 4  
-wx = 0 + 2 + 1 = 3  
--x = 0 + 0 + 1 = 1  
--- = 0 + 0 + 0 = 0

## Common chmod Examples

chmod 755 file.sh → rwxr-xr-x (file.sh)  
chmod 644 file.txt → rw-r--r-- (file.txt)  
chmod 700 script.sh → rwx----- (script.sh)  
chmod -R 755 dir/ → Apply 755 to all files in directory recursively

## Notes

- chmod changes the permission (mode) of files and directories.
- Use numeric mode (e.g., 755) or symbolic mode (e.g., u+x).
- Root user can override permissions.
- Be careful while using chmod -R on directories.

# GRANTING PERMISSION TO A FILE

| Permission | Symbol | Value |
|------------|--------|-------|
| Read       | 'r'    | 4 ✓   |
| Write      | 'w'    | 2 ✓   |
| Execute    | 'x'    | 1 ✓   |

① For (read + write + Execute)  $\Rightarrow 4 + 2 + 1 \Rightarrow 7$ .

② For No permission  $\Rightarrow 0$ .

## Examples:

```
~$ ls -lstr
```

$\Rightarrow$  drwxr-xr-x dummy (directory)

$\Rightarrow$  -rw-r--r-- script.sh (file)

## Permission string format

drwxr-xr-x

l  
d  
type

rwX  
user

r-x  
group

r-x  
others

## For files:

# chmod [number] filename

① ~\$ chmod 777 script.sh

-rwxrwxrwx script.sh (full access)

777

rwX rwX rwX

(user) (group) (others)

full read + write + execute access

② ~\$ chmod 755 script.sh

-rwxr-xr-x script.sh

755

rwX r-x r-x

(user) (group) (others)

user : read + write + execute  
group : read + execute  
others: read + execute

## For directories:

① ~\$ chmod -R 777 dummy

drwxrwxrwx dummy (full access)

## -R (Recursive)

Apply the permission to the directory and all its contents (files and subdirectories) recursively.

## Common Numeric Permissions

| Number | Permission | Meaning                                               |
|--------|------------|-------------------------------------------------------|
| 777    | rwxrwxrwx  | Full access for all                                   |
| 755    | rwxr-xr-x  | Full access to user, read & execute to group & others |
| 644    | rw-r--r--  | Read & write for user, read for group & others        |
| 600    | rw-----    | Read & write for user only                            |
| 400    | r-----     | Read for user only                                    |
| 0      | -----      | No permission                                         |

## Tips

- Use `ls -l` to see current permissions.
- `chmod` changes the mode (permission) of files and directories.
- Use numeric mode (e.g., 755) or symbolic mode (e.g., u+x, g-w).
- Be careful while using -R on directories.
- Root user can override permissions.